

Dynamic programming

COMPLETE LEARNING GUIDE

What You'll Learn

Dynamic programming (DP) systematically solves problems that exhibit overlapping subproblems (the same smaller problem is solved repeatedly) and optimal substructure (an optimal solution can be built from optimal solutions to its subproblems) by solving each distinct subproblem exactly once and reusing the result, turning what would otherwise be exponential brute-force recursion into polynomial time. This guide covers the two implementation styles — memoization (top-down, recursion plus a cache) and tabulation (bottom-up, filling a table iteratively) — through classic 1D examples (Fibonacci, climbing stairs) and classic 2D examples (0/1 knapsack, longest common subsequence), a general framework for identifying a DP state and its transition, and space-optimization techniques that reduce a DP table's memory footprint once you notice each state only depends on a limited window of previous states. DP is consistently ranked among the hardest interview topics, precisely because the hard part is recognizing the state and transition, not writing the loop once you have them.

1. Overlapping Subproblems and Optimal Substructure

A problem has overlapping subproblems if a naive recursive solution solves the identical smaller subproblem many times — naive Fibonacci recomputes `fib(2)` thousands of times for even moderate `n`, which is the direct signal that caching would help. A problem has optimal substructure if its optimal solution can be constructed directly from optimal solutions to its subproblems — the shortest path from A to C through B is the shortest path A-to-B plus the shortest path B-to-C, with no need to consider non-optimal sub-paths. Both properties together are DP's precondition: without overlapping subproblems, there's nothing to cache and gain from; without optimal substructure, combining subproblem answers wouldn't produce a correct overall answer.

2. Memoization: Top-Down DP

Memoization keeps the natural recursive structure of the brute-force solution but adds a cache (an array or `HashMap` keyed by the subproblem's parameters), checking the cache before doing any recursive work and storing the result before returning — this is usually the easiest DP style to derive, since you start from a correct (if slow) recursive brute force and add exactly one change: cache-check-then-store. Its main costs relative to tabulation are the overhead of recursive calls and, for deep recursion, call-stack space that a purely iterative tabulation approach avoids entirely.

JAVA

```
// Climbing Stairs: how many distinct ways to climb n stairs, taking 1 or 2 steps at a time
// Memoized (top-down) -- O(n) time, O(n) space (cache + call stack)
public static int climbStairs(int n, Map<Integer, Integer> cache) {
    if (n <= 2) return n;    // base cases: 1 way for 1 stair, 2 ways for 2 stairs
    if (cache.containsKey(n)) return cache.get(n);
    int ways = climbStairs(n - 1, cache) + climbStairs(n - 2, cache);
    cache.put(n, ways);
    return ways;
}
```

3. Tabulation: Bottom-Up DP

Tabulation builds a table iteratively from the smallest base-case subproblems up to the final answer, with no recursion at all — each table entry is computed once, directly from already-filled earlier entries, guaranteeing every subproblem is solved in a fixed, predictable order with no call-stack overhead. This style is typically the same time complexity as its memoized counterpart, but strictly better space complexity when recursion depth would otherwise be significant, and it's usually preferred in production and competitive settings for that reason, even though memoization is often easier to derive first.

JAVA

```
// Climbing Stairs -- Tabulated (bottom-up) -- O(n) time, O(n) space
public static int climbStairsTabulated(int n) {
    if (n <= 2) return n;
    int[] dp = new int[n + 1];
    dp[1] = 1;
    dp[2] = 2;
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];    // this state depends only on the
        // previous two
    }
    return dp[n];
}
```

4. Classic 2D DP: 0/1 Knapsack

The 0/1 knapsack problem asks: given items each with a weight and value, and a knapsack with a maximum weight capacity, which subset of items (each either fully included or fully excluded — no fractional items, hence "0/1") maximizes total value without exceeding capacity? The DP state $dp[i][w]$ represents "the maximum value achievable using only the first i items with capacity w ," and its transition considers two choices for item i : exclude it ($dp[i-1][w]$) or include it if it fits (its value plus $dp[i-1][w - \text{weight}[i]]$) — taking the better of the two.

JAVA

```
// 0/1 Knapsack -- O(n * capacity) time and space
public static int knapsack(int[] weights, int[] values, int capacity) {
    int n = weights.length;
    int[][] dp = new int[n + 1][capacity + 1];    // dp[i][w] = max value using
    first i items, capacity w

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i - 1][w];              // choice
1: exclude item i if (weights[i - 1] <= w) {
                dp[i][w] = Math.max(dp[i][w],
                    values[i - 1] + dp[i - 1][w - weights[i - 1]]); // choice
2: include item i
            }
        }
    }
    return dp[n][capacity];
}
```

5. Classic 2D DP: Longest Common Subsequence (LCS)

LCS asks for the length of the longest sequence of characters that appears (in order, not necessarily contiguously) in both of two given strings — used directly in diff tools and version control merge algorithms. The state $dp[i][j]$ represents "the LCS length of the first i characters of string A and the first j characters of string B," and the transition is: if the current characters match, extend the diagonal LCS by one ($dp[i-1][j-1] + 1$); if they don't match, take the best of skipping a character from either string ($\max(dp[i-1][j], dp[i][j-1])$).

JAVA

```
// Longest Common Subsequence -- O(m * n) time and space
public static int lcs(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1; // characters match
-- extend diagonally
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]); // skip a
character from either string
            }
        }
    }
    return dp[m][n];
}
```

6. A General Framework for Identifying State and Transition

Approaching an unfamiliar DP problem systematically means answering three questions in order: what does a state represent (usually one or two indices/parameters that fully describe a subproblem, like "first i items with capacity w "), what are the base cases (the smallest states, answerable directly with no further breakdown), and what is the transition (how does a state's answer derive from smaller/previous states — usually a small number of explicit choices, each evaluated and the best one kept). Writing this out explicitly in words before writing any code is the single highest-leverage habit for DP interview problems, since a correct state definition makes the code almost mechanical, while a wrong one makes debugging nearly impossible.

7. Space Optimization: Rolling Arrays

Many 2D DP solutions only ever read from the immediately previous row (or a small fixed window of previous rows/columns) to compute the current one — knapsack's $dp[i][w]$ only depends on row $i-1$, and LCS's $dp[i][j]$ only depends on row $i-1$ and the current row's already-computed $dp[i][j-1]$. Recognizing this lets you collapse a 2D table into one or two 1D arrays (a "rolling array"), reducing space from $O(n \cdot m)$ to $O(m)$, which is both a genuine performance improvement and a common interview follow-up question ("can you reduce the space complexity?") once a correct 2D solution is presented.

Common Pitfalls

- Jumping straight to code without first writing out the DP state and transition in words, leading to a table whose indices don't actually correspond to a well-defined subproblem.
- Confusing 0/1 knapsack (each item used at most once) with the unbounded knapsack variant (items can be reused), applying the wrong transition and iteration order.
- Off-by-one errors between a DP table's 1-indexed convention (common for cleanly handling the "zero items/characters" base case) and 0-indexed input arrays/strings.
- Missing that a correct memoized (top-down) solution and its tabulated (bottom-up) equivalent should have the same time complexity — if they differ, one of the two has a bug.
- Not attempting the space-optimization follow-up (rolling arrays) when explicitly asked, despite the 2D solution only ever needing the previous row.
- Applying a greedy approach to a problem that actually requires DP (like 0/1 knapsack, where greedy-by-value-density fails once items can't be fractionally split) — see the Greedy Algorithms submodule for exactly when greedy breaks down.
- Forgetting to initialize base cases in a tabulated solution correctly, producing subtly wrong results for small inputs even though the general transition logic is correct.

Key Takeaways

- DP applies when a problem has both overlapping subproblems (naive recursion repeats work) and optimal substructure (optimal answers compose from optimal subanswers).

- Memoization (top-down) adds a cache to a natural recursive solution; tabulation (bottom-up) fills a table iteratively — same time complexity, tabulation usually better space.
- Always define the DP state and transition explicitly in words before coding — this is the actual hard part of a DP problem, not the loop itself.
- 0/1 Knapsack and Longest Common Subsequence are the two most important 2D DP patterns to know cold — many other problems are variations on them.
- A DP state that only depends on the previous row (or a small window) can be space-optimized from $O(n \cdot m)$ to $O(m)$ with a rolling array.
- Greedy algorithms can fail where DP succeeds — knapsack is the textbook example of a problem greedy alone cannot solve correctly.

Learn More

GeeksforGeeks — Dynamic Programming

<https://www.geeksforgeeks.org/dsa/dynamic-programming/>

GeeksforGeeks — 0/1 Knapsack Problem

<https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/>

GeeksforGeeks — Longest Common Subsequence

<https://www.geeksforgeeks.org/dsa/longest-common-subsequence-dp-4/>